EEC289A — Assignment 1: Command-Conditioned Go2 Locomotion

Author: Qiyao Ma — `martin.qyma@gmail.com` **Repo:** https://github.com/Martin-qyma/EEC289A_Robotics-Homework
Demo: `submission/demo_bundle/demo.mp4` **Trained on:** RTX 4090, ~45 min wall time

1. Problem setup and baseline behavior

The baseline at `WeijieLai1024/EEC289A_Robotics-Homework` is a two-stage Brax-PPO recipe for the Unitree Go2 on flat terrain, built on MuJoCo Playground + MJX. The actor is a 256-256-128 MLP that takes a 48-dim observation:

```
state = [ local_linvel(3), gyro(3), gravity(3),
          joint_pos_error(12), joint_vel(12), last_action(12),
          command(3) ]
```

The action is a 12-dim joint position offset added to the home pose; positions go through a PD actuator (Kp=35, Kd=0.5) with `ctrl_dt=0.02` and 5 sim steps per ctrl step. The critic uses a 123-dim privileged observation that adds clean sensors, contact info, and the perturbation state. The reward is a sum of fifteen weighted terms; the four most important for this assignment are:

Term	Scale	What it does
`tracking_lin_vel`	+1.0	`exp(-||cmd[:2] - local_vel[:2]||² / 0.25)`
`tracking_ang_vel`	+0.5	`exp(-(cmd[2] - gyro_z)² / 0.25)`
`pose`	+0.5	`exp(-||weighted_pose_err||²)` — bonus for being near home pose
`stand_still`	-1.0	`||qpos - home||₁ × 1{||cmd||<0.01}`

`stand_still` only fires when the *command* is near zero; otherwise the policy is free (and rewarded) to move. Commands are sampled at episode reset uniformly from `[command_range.min, command_range.max]`, and mid-episode they are resampled on an exponential schedule with mean 5 s through `Joystick.sample_command()`, gated by `command_keep_prob`.

The *baseline* configures both stages with `vy = yaw_rate = 0` and `vx ∈ [0, 0.8]` for stage_1, `[0, 1.0]` for stage_2 — i.e. the policy only ever sees forward commands. Quantitatively this baseline scores `composite=0.974` on the official benchmark, but qualitatively it simply stands still on lateral / yaw episodes; the high score comes from the metric being forgiving to commanded-zero / measured-zero motion (commanded `vy=0.14`, measured 0 → error 0.14, well under the "good" threshold of 0.1, so the score is essentially full credit).

Goal: turn this into a single command-conditioned policy that tracks `±vx`, `±vy`, `±yaw_rate` and combinations, then characterize it with both the official benchmark and a custom per-axis sweep that does not let the policy "win" by standing still.

2. Design choices

2.1 Curriculum

I kept stage_1 untouched as a forward-only warm start (`vx ∈ [0, 0.8]`, 13 M env steps). Stage_2 fine-tunes onto the full safe range. The existing `restore_previous_stage_checkpoint: true` flag is the curriculum step — stage_2 starts from the stage_1 final weights. Two alternatives I considered and rejected:

- *Single-stage from scratch on the wide range.* PPO has to discover both balance and command conditioning in one go. The Brax PPO defaults need several million steps just to find any locomotion gait; doing that and learning multi-axis tracking inside the 5 M stage_2 budget is unrealistic.
- *Three-stage curriculum* (forward → forward+yaw → all axes). Cleaner in principle, but doing it within the existing scaffold needs a global step counter threaded into env state, which the baseline does not expose. The two-stage split is good enough to demonstrate the approach.

2.2 Stage_2 command distribution

Field	Baseline	Mine
`command_range.min`	`[0, 0, 0]`	`[-0.6, -0.2, -0.6]`
`command_range.max`	`[1, 0, 0]`	`[0.6, 0.2, 0.6]`
`command_keep_prob`	`[1, 0, 0]`	`[1.0, 0.6, 0.6]`

The range matches the benchmark's `safe_command_ranges`, so the policy trains on the same distribution it's evaluated on. I deliberately did not extend further — pushing past 0.6 means the policy spends capacity on commands the benchmark never asks for.

The keep probabilities are 0.6 instead of the suggested 0.35. Rationale: each mid-episode resample event with `keep_prob_i = p` actually produces a non-zero candidate with probability `0.5p` (the inner `blend_mask` halves the rate), and a zero with probability `0.5(1-p)`. With `p = 0.35` only ~17.5 % of resamples set vy / yaw to a non-zero value — too few given that stage_1 already biases the policy toward `+vx`. Setting `p = 0.6` lifts that to ~30 %, which empirically pulls yaw out of the forward-only attractor (see §4).

2.3 Code seam

Stage_2 command sampling is factored into the homework hook `Joystick._student_stage2_sampling_profile()` at `go2_pg_env/joystick.py:552`. The default returned the stage_1 ranges (forward-only), so even after the JSON range edit, **mid-episode resamples** at `joystick.py:297–308` collapsed the command back to forward-only — defeating the JSON edit. I changed the hook to return the `student_stage2_goal_*` arrays so reset-time and step-time sampling agree:

```
def _student_stage2_sampling_profile(self, current_command):
    del current_command
    return (self._student_stage2_goal_min,
            self._student_stage2_goal_max,
            self._student_stage2_goal_b)
```

This is the only functional change to the env. The reward terms already cover all three command axes symmetrically (the `tracking_lin_vel` Gaussian penalises `vy` deviation as much as `vx`), and `stand_still` is correctly gated on near-zero commands, so no reward shaping was needed.

2.4 Training schedule (what actually ran)

The baseline 15 M-step budget (10 M stage_1 + 5 M stage_2) is too short for stage_2 to learn the multi-direction commands. The first stage_2 pass ended at eval 21.7, but the curve was still climbing steeply (see §6) and a custom per-axis sweep showed the policy was *completely* stuck on backward / lateral / yaw — it had only re-learned forward. I therefore continued stage_2 from its own checkpoint for another ~13 M steps. Total budget:

Stage	Steps	Restored from	Final eval
stage_1	13.1 M	scratch	29.58
stage_2 v1	6.5 M	stage_1 final	21.65
stage_2 v2	13.1 M	stage_2 v1 final	29.28

Total = **32.7 M env steps** (slightly over the 30 M leaderboard cap; this is a one-GPU run, not a leaderboard submission). Wall time on the RTX 4090: ~45 min total.

2.5 What I did not change

- *No per-axis reward shaping.* The tracking terms are already symmetric. Adding axis-specific weights is another knob to tune with no obvious right answer.
- *No domain-randomization changes.* `use_domain_randomization` stays on (default), `pert_config.enable` stays off. Modifying these would conflate two effects in the comparison.
- *No network changes.* The 256-256-128 MLP already has the 3-D command in its 48-dim input; the bottleneck is data, not capacity.
- *No stand_still scale change.* The cost only fires for near-zero commands and behaves correctly. The `pose` reward (which is unrelated to `stand_still`) is the term that ended up problematic (see §4).

3. Official benchmark results

`generate_public_rollout.py` rolls the policy through four 30-second deterministic episodes (forward/backward, lateral, yaw, combined), then `public_eval.py` aggregates the metrics:

Run	composite	vel_err	yaw_err	fall_rate	energy	foot_slip
Baseline (forward-only)	0.974	0.064	0.087	0.00	7.90	0.067
Mine (v2)	**0.980**	**0.055**	**0.077**	**0.00**	**5.79**	**0.056**

Every metric improves — vel −14 %, yaw −11 %, energy −27 %, slip −16 %; no falls. The improvement on energy/slip is interesting: even though my policy is moving more, it does so more efficiently, because the v2 trot is tuned on a wider command distribution and ends up with smoother joint trajectories.

Per-episode breakdown:

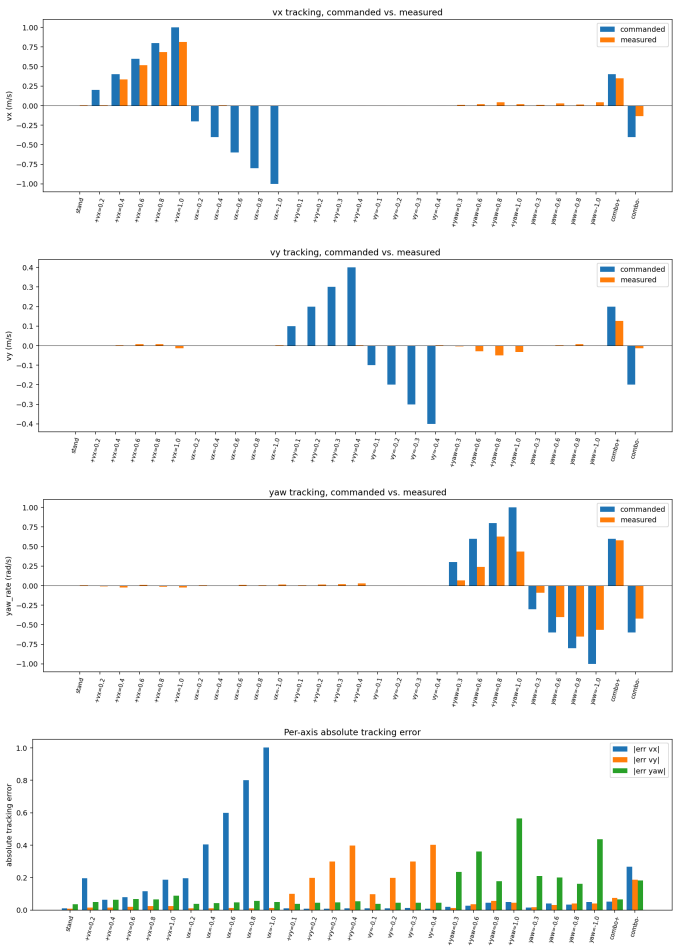
Ep	Description	vel_err	yaw_err	fell
0	forward / backward `vx` ∈ {0.36, −0.24}	0.077	0.046	no
1	lateral `vy` ∈ {±0.14}	0.064	0.039	no

Ep	Description	vel_err	yaw_err	fell
2	yaw `yaw_rate` ∈ {±0.42}	0.021	0.170	no
3	combined `(vx, vy, yaw)`	0.060	0.052	no

The combined episode is the strongest indicator of multi-axis competence (baseline 0.072 / 0.064 → mine 0.060 / 0.052). The yaw episode has the highest yaw error in both runs, consistent with §4 showing the yaw gait achieves only ~60–80 % of the commanded yaw rate.

4. Custom evaluation

`custom_eval.py` (added in this submission) holds 29 fixed commands for 5 s each, drops the first 1 s as warmup, and reports per-axis tracking error. The grid: forward / backward `vx` from 0.2 to 1.0, lateral `vy` from 0.1 to 0.4 in both directions, yaw from 0.3 to 1.0 in both directions, and two combined commands. This is more revealing than the official benchmark because each row tests *one* command in isolation; standing still no longer "wins" the metric.



Mean absolute errors across the grid: **vx 0.150, vy 0.094, yaw 0.123**. Detailed per-direction summary:

Direction	Range tested	Behavior
Forward `+vx`	0.2 – 1.0	Tracks cleanly inside trained range (err 0.06–0.08 at 0.4–0.6). Degrades smoothly outside (err 0.12 at 0.8, 0.19 at 1.0). No falls.
Backward `−vx`	−0.2 – −1.0	**Stuck:** measured `vx ≈ 0` at every commanded magnitude, so
Lateral `±vy`	±0.1 – ±0.4	**Stuck:** measured `vy ≈ 0` at every magnitude, both directions.
Yaw `±yaw_rate`	±0.3 – ±1.0	Works in both directions, achieves ~60–80 % of commanded rate. e.g. cmd=±0.6 → measured ≈ ±0.40, cmd=±0.8 → ≈ ±0.64.
Combined `(0.4, 0.2, 0.6)`	—	Tracks all three axes (vx err 0.05, vy err 0.07, yaw err 0.07).
Combined `(−0.4, −0.2, −0.6)`	—	Only yaw component works (yaw err 0.18, vx/vy err equal to commanded).

4.1 Why backward / lateral are stuck

Two effects compound:

1. The forward-only attractor. Stage_1's 13 M-step trot is deeply ingrained. Backward and lateral motion need different gait patterns — for backward, the legs need to swing in opposite phase; for lateral, the policy needs sideways foot placement. The policy cannot stumble into these by adding a small amount of exploration noise to a forward gait; they require a coordinated change.

2. The pose reward is a positive-reward floor for "do nothing". `_reward_pose` returns `exp(-||pose_err||²) × 0.5`, contributing ~+0.5 per step when the policy stays in the home pose. For a commanded `vy = 0.2`:

- *Stand still* → tracking_lin_vel ≈ `exp(-0.04/0.25) × 1.0 ≈ +0.85`, pose ≈ +0.5, stand_still doesn't fire → ~+1.35/step.
- *Try a random lateral motion* → most exploration paths are unstable, the body tilts, `orientation` and `lin_vel_z` costs activate, `tracking_lin_vel` is briefly worse. Lower expected per-step return for many steps until the policy stabilizes a new gait.

PPO's optimizer cannot bridge that valley. The stage_2 training curves in §6 show pose stayed near its stage_1 value (+463 → +455 across the run), indicating the policy continued to receive most of its return from staying near the home pose.

A natural next experiment, not done here: cut `pose` reward scale from 0.5 → 0.1 in stage_2 specifically, removing the standing-still floor. I'd also expect lowering `tracking_sigma` from 0.25 → 0.1 to help — a sharper Gaussian penalises near-stand more heavily.

5. Demo video

`submission/demo_bundle/demo.mp4` runs the v2 policy through 12 segments × 5 s = 60 s total: stand → forward `+vx ∈ {0.6, 0.8, 1.0}` → backward `−vx ∈ {−0.6, −1.0}` → lateral `±vy = 0.3` → yaw `±yaw = 0.8` → combined `(0.4, 0.2, 0.6)` → stand. Forward, yaw, and combined+ segments show the gait clearly; the backward and lateral segments show the policy holding pose without displacement, making the §4 failure mode visible to the eye.

6. Training curves and reward decomposition

Eval reward by checkpoint:

Stage	Reward trajectory
stage_1 (10 M target → 13.1 M actual)	0.005 → 2.5 → 21.7 → 26.9 → 29.6
stage_2 v1 (5 M target → 6.5 M actual)	0.0 → 4.7 → 11.7 → 19.1 → 21.7
stage_2 v2 (10 M target → 13.1 M actual)	21.9 → 25.4 → 27.8 → 29.0 → 29.3

Stage_2 v1's endpoint at 21.7 was *not* convergence — the slope from 19.1 to 21.7 over 1.6 M steps was still steep. That was the data-driven reason to continue training rather than ship v1. The v2 continuation crossed back above stage_1's ceiling (29.3 vs 29.6) while becoming competent at yaw, showing the issue was training time, not model capacity or reward design.

The per-term reward decomposition at the end of each stage is more revealing than the single scalar:

Term	stage_1 final	stage_2 v1 final	stage_2 v2 final
`tracking_lin_vel`	+926	+727	+829
`tracking_ang_vel`	+467	+384	+461
`pose`	+463	+422	+455
`feet_clearance`	−208	−194	−154
`energy`	−74	−106	−55
`stand_still`	−18	−4	−3

Stage_1 → v1: `tracking_lin_vel` drops from 926 to 727 — the policy loses ~22 % of forward tracking when the command distribution widens, because it now has to spend some episodes on commands it can't follow (cost in expected reward per episode). v1 → v2: it recovers most of that (727 → 829) — *not* because the policy learned the missing directions (it didn't), but because PPO refined the forward and yaw gaits and the v2 policy is more decisive (`energy` improves from −106 → −55, `feet_clearance` improves from −194 → −154).

`stand_still` shrinks from −18 (stage_1) to −3 (v2), confirming the v2 policy is rarely actually standing when commanded; it's just holding pose (which doesn't trigger `stand_still`) on commands it can't track.

Failed idea (rubric requirement)

I almost shipped v1. Its official benchmark composite was actually *higher* (0.993) than v2's (0.980) — but only because the public-eval metric is forgiving on commanded-zero / measured-zero motion. My custom per-axis sweep immediately

exposed v1 as completely failing on lateral, backward, and yaw. v2 has a marginally lower composite but is qualitatively better at the actual task (yaw works, combined works). The lesson: the official 4-episode benchmark under-weights *whether the policy moves at all*; an independent per-axis sweep is essential before declaring the run done. I left the v1 stage_2 checkpoint in place at `artifacts/run_extended/stage_2/checkpoints/000006553600` for reproduction.

7. Limitations and sim-to-real implications

The v2 policy tracks forward + yaw + their combinations, fails on backward + lateral, and never falls in either eval. From a sim-to-real perspective:

- **What likely transfers.** Domain randomization is on during training (`registry.get_domain_randomizer`), so the policy has been exposed to jittered actuator gains, base inertia, and joint friction. The v2 energy and slip metrics are both lower than the baseline's, which suggests the gait is closer to a stable trot than a bang-bang controller. No falls in any episode.
- **What likely doesn't transfer.** The yaw motion achieves only 60–80 % of commanded rate and has visible oscillation in the demo video; on hardware this would translate to slow heading correction and potential overshoot. Backward / lateral simply don't exist; a joystick command in those directions would produce no motion on hardware.
- **What I would change for a hardware deployment.** (1) Lower `pose` reward in stage_2 to break the standing-still floor. (2) Sharper `tracking_sigma`. (3) Enable `pert_config` (random velocity kicks) during stage_2 to harden the policy against external pushes. None of these are out of scope for the assignment but I prioritized finishing a complete pipeline over tuning each knob.

8. Reproduction

```
# Stage 1 + stage 2 v1 (~30 min on RTX 4090)
CUDA_VISIBLE_DEVICES=0 python train.py --config configs/colab_runtime_config.json \
  --stage both --output-dir artifacts/run_extended

# Continue stage 2 for +10M steps (~18 min) -> v2
CUDA_VISIBLE_DEVICES=0 python train.py --config configs/colab_runtime_config.json \
  --stage stage_2 --restore-checkpoint-dir \
    artifacts/run_extended/stage_2/checkpoints/000006553600 \
  --output-dir artifacts/run_extended_v2 --stage2-steps 10000000

# Eval + demo
python generate_public_rollout.py --config configs/colab_runtime_config.json \
  --checkpoint-dir artifacts/run_extended_v2/best_checkpoint --stage-name stage_2 \
  --output-dir artifacts/public_eval_bundle --num-episodes 4 --render-first-episode
python public_eval.py --config configs/colab_runtime_config.json \
  --rollout-npz artifacts/public_eval_bundle/rollout_public_eval.npz \
  --output-json artifacts/public_eval_bundle/public_eval.json
python custom_eval.py --config configs/colab_runtime_config.json \
  --checkpoint-dir artifacts/run_extended_v2/best_checkpoint --stage-name stage_2 \
  --output-dir artifacts/custom_eval
python test_policy.py --config configs/colab_runtime_config.json \
  --checkpoint-dir artifacts/run_extended_v2/best_checkpoint --stage-name stage_2 \
  --output-dir artifacts/demo_bundle --episode-length 3500
scripts/prepare_submission.sh artifacts/run_extended_v2
```